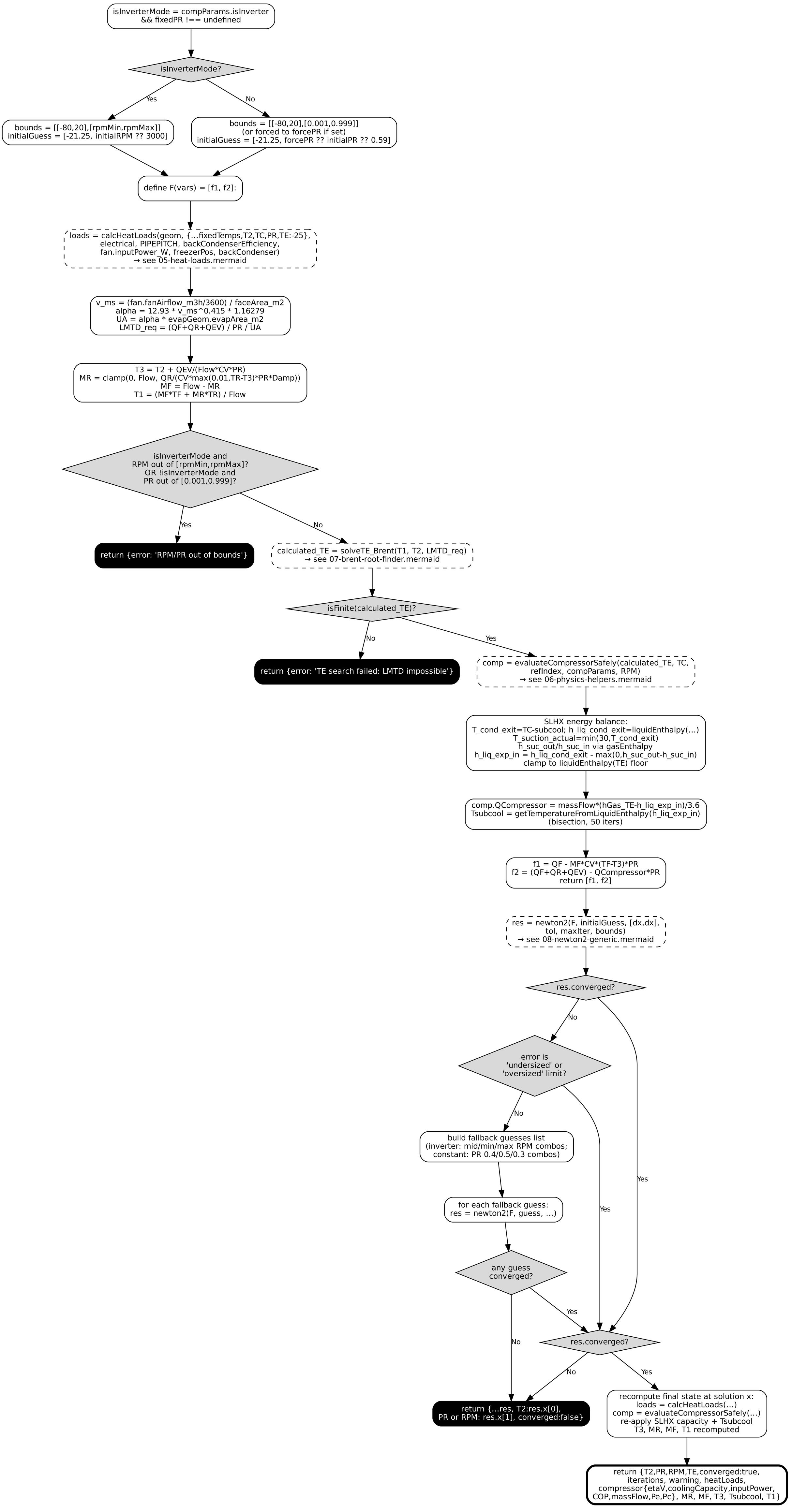
isInverterMode = compParams.isInverter
&& fixedPR !== undefined
isInverterMode?
Yes
No
bounds = [[-80,20],[rpmMin,rpmMax]]
initialGuess = [-21.25, initialRPM ?? 3000]
bounds = [[-80,20],[0.001,0.999]]
(or forced to forcePR if set)
initialGuess = [-21.25, forcePR ?? initialPR ?? 0.59]
define F(vars) = [f1, f2]:
loads = calcHeatLoads(geom, {...fixedTemps,T2,TC,PR,TE:-25},
electrical, PIPEPITCH, backCondenserEfficiency,
fan.inputPower_W, freezerPos, backCondenser)
→ see 05-heat-loads.mermaid
v_ms = (fan.fanAirflow_m3h/3600) / faceArea_m2
alpha = 12.93 * v_ms^0.415 * 1.16279
UA = alpha * evapGeom.evapArea_m2
LMTD_req = (QF+QR+QEV) / PR / UA
T3 = T2 + QEV/(Flow*CV*PR)
MR = clamp(0, Flow, QR/(CV*max(0.01,TR-T3)*PR*Damp))
MF = Flow - MR
T1 = (MF*TF + MR*TR) / Flow
isInverterMode and
RPM out of [rpmMin,rpmMax]?
OR !isInverterMode and
PR out of [0.001,0.999]?
Yes
No
return {error: 'RPM/PR out of bounds'}
calculated_TE = solveTE_Brent(T1, T2, LMTD_req)
→ see 07-brent-root-finder.mermaid
isFinite(calculated_TE)?
No
Yes
return {error: 'TE search failed: LMTD impossible'}
comp = evaluateCompressorSafely(calculated_TE, TC,
refIndex, compParams, RPM)
→ see 06-physics-helpers.mermaid
SLHX energy balance:
T_cond_exit=TC-subcool; h_liq_cond_exit=liquidEnthalpy(...)
T_suction_actual=min(30,T_cond_exit)
h_suc_out/h_suc_in via gasEnthalpy
h_liq_exp_in = h_liq_cond_exit - max(0,h_suc_out-h_suc_in)
clamp to liquidEnthalpy(TE) floor
comp.QCompressor = massFlow*(hGas_TE-h_liq_exp_in)/3.6
Tsubcool = getTemperatureFromLiquidEnthalpy(h_liq_exp_in)
(bisection, 50 iters)
f1 = QF - MF*CV*(TF-T3)*PR
f2 = (QF+QR+QEV) - QCompressor*PR
return [f1, f2]
res = newton2(F, initialGuess, [dx,dx],
tol, maxIter, bounds)
→ see 08-newton2-generic.mermaid
res.converged?
No
Yes
error is
'undersized' or
'oversized' limit?
No
Yes
build fallback guesses list
(inverter: mid/min/max RPM combos;
constant: PR 0.4/0.5/0.3 combos)
for each fallback guess:
res = newton2(F, guess, ...)
any guess
converged?
No
Yes
res.converged?
No
Yes
return {...res, T2:res.x[0],
PR or RPM: res.x[1], converged:false}
recompute final state at solution x:
loads = calcHeatLoads(...)
comp = evaluateCompressorSafely(...)
re-apply SLHX capacity + Tsubcool
T3, MR, MF, T1 recomputed
return {T2,PR,RPM,TE,converged:true,
iterations, warning, heatLoads,
compressor{etaV,coolingCapacity,inputPower,
COP,massFlow,Pe,Pc}, MR, MF, T3, Tsubcool, T1}